

Chat History: EviVLM Investigation

User: refer the paper

@EviVLM_When_Evidential_Learning_Meets_Vision_Language_Model_for_Medical_Image_Segmentation (1).pdf . where can i find datasets to train this VLM?

Assistant: Based on the paper "EviVLM: When Evidential Learning Meets Vision Language Model for Medical Image Segmentation," the model was trained and evaluated on several public medical imaging datasets. You can find them at the following locations:

1. QaTa-COV19 Dataset

- **Description:** Consists of 9,258 COVID-19 chest X-ray radiographs.
- **Source:** Compiled by researchers from Qatar University and Tampere University.
- **Text Annotations:** The text prompts used for this dataset were originally provided by the [LViT paper \(Li et al., 2024\)](#).
- **Link:** [QaTa-COV19 on Kaggle](#) or via the [official project page](#).

2. MosMedData+ Dataset

- **Description:** Contains 2,729 CT scan slices of lung infections.
- **Source:** A combination of the MosMedData and other COVID-19 lesion masks.
- **Text Annotations:** Similar to QaTa-COV19, the text annotations are sourced from the [LViT project](#).
- **Link:** [MosMedData Official Site](#) or [GitHub Repository](#).

3. Duke-Breast-Cancer-MRI Dataset

- **Description:** Contains 922 MRI scans from 922 breast cancer patients with verified text annotations.
- **Link:** Available via [The Cancer Imaging Archive \(TCIA\)](#).

4. Other Evaluated Datasets

The paper also mentions performance on:

- **BraTS 2020:** A 3D dataset for brain tumor segmentation. Available at the [official BraTS challenge site](#).
- **MoNuSeg:** Used for nuclear segmentation in pathology. Available on [Grand Challenge](#).

Pro-Tip: Check the Official Codebase

The authors provide their implementation on GitHub, which often contains specific scripts or links for data preparation:

<https://github.com/QingtaoPan/EviVLM>

User: i have gone through the datasets, it seems they lack in text descriptions which is essential for a VLM to be trained.

Assistant: The text descriptions you are looking for are indeed not part of the raw medical imaging datasets found on platforms like Kaggle or TCIA. For Vision-Language Models (VLMs) like EviVLM, these descriptions are typically provided as **supplementary "language-enhanced" annotations** by the researchers.

Here is where you can find the specific text files required for each dataset mentioned in the paper:

1. QaTa-COV19 & MosMedData+

The text annotations for these datasets were originally created for the **LViT paper** (Li et al., 2024). You can find the Excel files (`Train_text.xlsx` , `Test_text.xlsx`) containing the "Image ID" and "Description" mappings in their official repository:

- **Source:** [LViT GitHub Repository \(HUANGLIZI/LViT\)](#)
- **Files:** Check the `datasets/` folder in that repository for `.xlsx` files paired with each dataset.

2. Duke-Breast-Cancer-MRI

The text descriptions for this dataset were curated by the EviVLM authors themselves. They are typically included in the EviVLM codebase or provided as a download link in the repository:

- **Source:** [EviVLM GitHub Repository \(QingtaoPan/EviVLM\)](#)
- **Format:** Look for files like `Train_text.xlsx` or JSON mappings that describe tumor characteristics (location, shape, size) for specific MRI slices.

3. MoNuSeg

The paper explicitly mentions that for MoNuSeg, the researchers **augmented simple text annotations using ChatGPT** to make them more descriptive. For example, a simple note like "The nuclei are evenly distributed" was expanded into:

"The nuclei are uniformly distributed across the field, showing consistent size, shape, and staining intensity without evident clustering or sparse regions."

Summary Checklist for Training

To train the model, your dataset folder structure should look like this (as expected by `code/train.py`):

```
|— Train_Folder
|   |— images/
|   |— labels/
|   |— Train_text.xlsx  <-- (From LViT/EviVLM repos)
|— Test_Folder
|   |— images/
|   |— labels/
|   |— Test_text.xlsx   <-- (From LViT/EviVLM repos)
```

User: the folder @code/** that you are seeing is a raw implementation of EviVLM. since this is a VLM first we have to make it speak english in order for it to understand text description of medical images... how do we train it? refer the paper @EviVLM_When_Evidential_Learning_Meets_Vision_Language_Model_for_Medical_Image_Segmentation (1).pdf see if they said anything on how they trained the model.

Assistant: According to the paper and the provided implementation, you don't need to "make it speak English" from scratch. The model follows a **Transfer Learning** strategy by using specialized, pre-trained weights for both vision and language.

Here is how the training works based on the paper and the @code files:

1. The "Language" Knowledge (BioClinicalBERT)

The model gains its understanding of medical English by using **BioClinicalBERT** as its text encoder (`code/backbones/bert_model/TextEncoder.py`).

- **Pre-training:** BioClinicalBERT was already trained on the MIMIC-III database (millions of clinical notes). This provides the "medical English" foundation.
- **Implementation:** In `TextEncoder.py` , you'll see it loads these weights from a local path (which you will need to download or point to):

```
self.Bio_ClinicalBERT_path = 'path/to/Bio_ClinicalBERT'
self.model = BertModel.from_pretrained(self.Bio_ClinicalBERT_path, ..
```

2. The Training Paradigm (End-to-End)

The paper (Section I, Page 8) specifies that EviVLM is trained **end-to-end** using a combination of three specific losses. You don't train the language and vision parts separately; you train them together to "align."

The Total Loss ($\mathcal{L}_{\text{overall}}$):

$$\mathcal{L}_{\text{overall}} = \omega_1 \mathcal{L}_{\text{edsl}} + \omega_2 \mathcal{L}_{\text{evi}} + \mathcal{L}_{\text{seg}}$$

- $\mathcal{L}_{\text{seg}}$ (**Segmentation Loss**): Standard Cross-Entropy/Dice loss between the predicted mask and the Ground Truth.
- $\mathcal{L}_{\text{edsl}}$ (**Evidence Differential Similarity Learning**): This is the "alignment" part. It forces the model to make the image features and text features consistent by measuring their "variation inconsistency."
- $\mathcal{L}_{\text{evi}}$ (**Evidential Loss**): This uses **Subjective Logic** to quantify the "modality gap" (the uncertainty between what the image sees and what the text says).

3. How to Execute Training

To train the model using your implementation, follow these steps based on `code/train.py`:

1. **Configure Paths:** Edit `code/Config_SSL.py` to point to your dataset folders and the pre-trained BERT/ViT weights.
2. **Hyperparameters:** The paper uses Adam optimizer (`lr = 3e-5`), Cosine Annealing scheduler, batch size 32, and up to 1000 epochs.
3. **Run the Script:** `python code/train.py`

User: so it uses a pretrained bert model to understand the text and learn descriptions, currently i am having a nvidia gtx 1650 laptop gpu, if i were to start the training with the datasets from <https://github.com/HUANGLIHUAN/LViT>, is it possible for me to train this VLM with my current specs, if yes then how much days/hours will it take? if no then what is the recommended specs?

Assistant: Training this specific model on an **NVIDIA GTX 1650 (4GB VRAM)** will be extremely challenging and, with the default settings, **virtually impossible**.

1. The VRAM Bottleneck (The "No" Part)

- **Model Weight Size:** BioClinicalBERT + DeiT-Base already take up nearly 1GB.
- **The "Batch Size 32" Problem:** The default batch size would require roughly **16GB to 20GB of VRAM**. Your GTX 1650 has only **4GB**.
- **The "Modality Gap" Modules:** Memory-intensive matrices will likely trigger "Out of Memory" (OOM) errors immediately.

2. Estimated Time (If it runs)

If you manage to get it running with a batch size of 1:

- **Per Epoch:** Likely **1 to 2 hours**.
- **Total Time:** Up to **40 to 60 days** of continuous training.

3. Recommended Specs

The paper explicitly states they used an **NVIDIA V100 (24GB VRAM)**.

- **Minimum:** NVIDIA RTX 3060 (12GB VRAM).
 - **Ideal:** NVIDIA RTX 3090 / 4090 (24GB VRAM).
 - **Free Alternative:** Use **Kaggle Kernels** (16GB VRAM) or **Google Colab**.
-

User: okay, i have a rtx 3060 pc in my college, how much hours/days will it take to train the VLM? what changes should i make in the configuration in order to run it smoothly?

Assistant: With an **RTX 3060 (12GB VRAM)**, you are in a much better position.

1. Estimated Training Time

- **Per Epoch:** Approximately **5 to 8 minutes**.
- **Total Time:** **12 to 18 hours** for full convergence (~150-200 epochs).

2. Required Configuration Changes

- **Edit `code/Config_SSL.py`:** Reduce `batch_size` to **8**. Update dataset paths.
 - **Edit Weight Paths:** In `TextEncoder.py` and `ImageEncoder.py`, update the hardcoded paths to your local weight files.
 - **Gradient Accumulation:** Use `accumulation_steps = 4` to simulate the paper's batch size of 32 (8×4).
-

User: if i were to do it in kaggle notebook, what gpu they provide and how many hours?

Assistant: Kaggle provides a **Single NVIDIA Tesla T4 (16GB VRAM)**.

1. Kaggle Specs & Limits

- **VRAM:** 16GB (Allows batch size **12 to 16**).
- **Weekly Quota:** 30 hours.
- **Session Limit:** 12 hours.

2. Estimated Training Time on Kaggle

- **Per Epoch:** ~6 to 10 minutes.
- **To Convergence:** 12 to 16 hours.

Final Verdict: Use **Kaggle**. The extra 4GB of VRAM over the RTX 3060 is critical for the stability of the alignment modules.